Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün.

Laboratoriya işi № 1

Tələbənin adı və soyadı: Nihad Fetullayev

Qrup: KOM25(A)

Müəllim: Şamil Hümbətov

Tarix: 12 oktyabr 2026

2026

Singly Linked List-in reallaşdırılması və tətbiqi

Məqsəd

Bu laboratoriya işinin məqsədi:

- Linked List verilənlər strukturunun iş prinsipini öyrənmək;
- Düyün (Node) anlayışını mənimsəmək;
- Siyahıya element əlavə etmə və silmə əməliyyatlarını reallaşdırmaq;
- Linked List və adi massiv (array/list) arasındakı fərqləri anlamaq.

Nəzəri hissə

Linked List düyünlərdən ibarətdir.

Hər düyün iki hissədən ibarətdir:

+-------+------+

| data | next |

+-------+------+

məsələn

10 -> 20 -> 30 -> NULL

Burada:

- `10`, `20`, `30` verilənlərdir
- `next` növbəti düyünün ünvanını göstərir
- `NULL` siyahının sonunu bildirir

Tapşırıq 1. Node sinfinin yaradılması

Aşağıdakı strukturu yaradın:

```
class Node:
    def init(self, data):
        self.data = data
        self.next = None
```

Test:

Node(15)

```
node = Node(15)
print(node.data)
print(node.next)
```

just experimenting with nodes and learning how they work, here in this 1ST task we just test with the node system I guess idrk what to say its a simple operation afterall...

Tapşırıq 2. Siyahının sonuna element əlavə edilməsi

Aşağıdakı siyahını formalaşdırın:

10 → 20 → 30 → 40

və çap edin.

Nəticə:

10 -> 20 -> 30 -> 40 -> None

```
    def append(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node

    def print_list(self):
        current = self.head
        while current is not None:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

my_list = LinkedList()
my_list.append(10)
my_list.append(20)
my_list.append(30)
my_list.append(40)
my_list.print_list()
```

IN this task we learn how to make a list of nodes and print them out while making sure the nodes start from 10 (make it head node)

Tapşırıq 3. Siyahının əvvəlinə element əlavə edilməsi

Verilmiş siyahı:

20 → 30 → 40

başlanğıca 10 əlavə edin.

Nəticə:

10 → 20 → 30 → 40

```
def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
self.head = new_node
```

this task teaches us how to insert a node at the beginning of the node list (pretty self explanetory if u ask me, cuz thats all to say about this task

Tapşırıq 4. Verilmiş elementdən sonra əlavə etmə

Verilmiş siyahı:

10 → 20 → 40

20-dən sonra 30 əlavə edin.

Nəticə:

10 → 20 → 30 → 40

```
def insert_after(self, target, data):
        current = self.head
        while current is not None and current.data != target:
            current = current.next
        if current is None:
            print("Element tapılmadı.")
            return
        new_node = Node(data)
        new_node.next = current.next
current.next = new_node
list_4 = LinkedList()
for number in (10, 20, 40):
    list_4.append(number)
list_4.insert_after(20, 30)
list_4.print_list()
```

this task teaches us how to insert a node after a specific number in the node system.

Tapşırıq 5. Element silmə

Verilmiş siyahı:

10 → 20 → 30 → 40 → 50

30 elementini silin.

Nəticə:

10 → 20 → 40 → 50

```
def delete(self, value):
        if self.head is None:
            print("Element tapılmadı.")
            return
        if self.head.data == value:
            self.head = self.head.next
            return
        current = self.head
        while current.next is not None and current.next.data != value:
            current = current.next
        if current.next is None:
            print("Element tapılmadı.")
            return
current.next = current.next.next
list_5 = LinkedList()
for number in (10, 20, 30, 40, 50):
    list_5.append(number)
list_5.delete(30)
list_5.print_list()
```

deletes the specific node. Need I say more? Im not gonna just yap about these I like to keep it simple and short (and because I don’t see in any way shape or form on how u can extend the explanations...)

Tapşırıq 6. Axtarış

Funksiya yazın:

search(value)

Əgər element siyahıda varsa:

Element tapıldı.

əks halda

Element tapılmadı.

```
def search(self, value):
        current = self.head
        while current is not None:
            if current.data == value:
                print("Element tapıldı.")
                return
```

```
            current = current.next
        print("Element tapılmadı.")

list_6 = LinkedList()
for number in (10, 20, 30, 40):
    list_6.append(number)
list_6.search(30)
list_6.search(99)
```

you give it a number. If it exists in the node list it returns found else not found.

Tapşırıq 7. Uzunluğun hesablanması

Linked List-də neçə element olduğunu hesablayın.

Misal:

11 → 22 → 33 → 44 → 55

Nəticə:

```
List length = 5
def length(self):
        count = 0
        current = self.head
        while current is not None:
            count += 1
            current = current.next
return count
```

counts the amount of individual nodes in the list.

Tapşırıq 8. Maksimum və minimum element

Aşağıdakı siyahı üçün:

23 → 8 → 91 → 15 → 42

tapın:

- maksimum element
- minimum element

```
def find_max_min(self):
        if self.head is None:
            return None, None
        maximum = self.head.data
        minimum = self.head.data
```

```
        current = self.head.next
        while current is not None:
            if current.data > maximum:
                maximum = current.data
            if current.data < minimum:
                minimum = current.data
                current = current.next
        return maximum, minimum
```

finds the max and min node in the list.

Tapşırıq 9. Siyahını tərs çevirmək

Verilmiş:

10 → 20 → 30 → 40 → 50

Nəticə:

50 → 40 → 30 → 20 → 10

Məhdudiyyət:

❌ Yeni Linked List yaratmaq olmaz.

```
def reverse(self):
        previous = None
        current = self.head
        while current is not None:
            next_node = current.next
            current.next = previous
            previous = current
            current = next_node
self.head = previous
```

pretty obvious… flips the list to make it reversed dunno whats there to explain/write much about this simple code command

Tapşırıq 10 (Challenge)

Tələbə qeydiyyat sistemi hazırlayın.

Hər düyündə:

student_id

student_name

gpa

saxlanılsın.

Aşağıdakı əməliyyatları təmin edin:

1. Tələbə əlavə et
2. Tələbə sil
3. Bütün tələbələri göstər
4. GPA > 80 olanları göstər
5. Ada görə axtarış et
6. Siyahıdakı tələbələrin sayını göstər

```
class StudentNode:
    def __init__(self, student_id, student_name, gpa):
        self.student_id = student_id
        self.student_name = student_name
        self.gpa = gpa
        self.next = None
def print_student(student):
    print(f"ID: {student.student_id}, Name: {student.student_name}, GPA: {student.gpa}")
class StudentList:
    def __init__(self):
        self.head = None
    def add_student(self, student_id, student_name, gpa):
        new_student = StudentNode(student_id, student_name, gpa)
        if self.head is None:
            self.head = new_student
            return
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_student
    def remove_student(self, student_id):
        if self.head is None:
            print("Tələbə tapılmadı.")
            return
        if self.head.student_id == student_id:
            self.head = self.head.next
            return
```

```
        current = self.head
        while current.next is not None and current.next.student_id !=
student_id:
            current = current.next
        if current.next is None:
            print("Tələbə tapılmadı.")
            return
        current.next = current.next.next
    def show_all(self):
        current = self.head
        while current is not None:
            print_student(current)
            current = current.next
    def show_high_gpa(self):
        current = self.head
        while current is not None:
            if current.gpa > 80:
                print_student(current)
            current = current.next
    def search_by_name(self, name):
        current = self.head
        while current is not None:
            if current.student_name.lower() == name.lower():
                print_student(current)
                return
            current = current.next
        print("Tələbə tapılmadı.")
    def count_students(self):
        count = 0
        current = self.head
        while current is not None:
            count += 1
            current = current.next
        return count
students = StudentList()
```

```
students.add_student(1, "Ali", 85)
students.add_student(2, "Leyla", 92)
students.add_student(3, "Murad", 74)
students.add_student(4, "Aysel", 88)
print("All students:")
students.show_all()
print("GPA above 80:")
students.show_high_gpa()
print("Search by name (Leyla):")
students.search_by_name("Leyla")
students.remove_student(3)
print("After removing ID 3:")
students.show_all()
print("Student count:", students.count_students())
```

this code basically adds up together all the previous functions we learned and combines them into 1 task

Bonus tapşırıq (+10 bal)

Singly Linked List və Doubly Linked List müqayisəsi

Aşağıdakıları araşdırın:

- Struktur fərqləri
- Üstünlüklər
- Çatışmazlıqlar
- Yaddaş sərfi
- İrəli və geri traversing

Nəticələri 1 səhifəlik qısa hesabat şəklində təqdim edin.

Qiymətləndirmə

- Tapşırıq 1-3: 20 bal
- Tapşırıq 4-5: 20 bal
- Tapşırıq 6-7: 15 bal
- Tapşırıq 8-9: 25 bal
- Tapşırıq 10: 20 bal

Bonus: +10 bal